

Tipos Abstractos de Datos (TAD)

Ejercicio 1 — TAD Fecha

Define un tipo abstracto de datos Fecha que represente una fecha del calendario mediante tres campos enteros: día, mes y año (usa typedef struct). Implementa las siguientes funciones sobre el TAD:

- `es_bisiesto(Fecha f)`: devuelve 1 si el año de `f` es bisiesto (divisible entre 4 y no entre 100, salvo que también sea divisible entre 400) y 0 en caso contrario.
- `es_valida(Fecha f)`: devuelve 1 si la fecha es válida (día y mes dentro de rango, y el día no supera el número de días del mes correspondiente) y 0 en caso contrario.

Escribe un programa principal que cree la fecha 29/2/2024, compruebe si es bisiesta y si es válida, y muestre el resultado por pantalla.

Ejercicio 2 — Comparación de fechas

Amplía el TAD Fecha del ejercicio 1 añadiendo la función:

- `comparar_fechas(Fecha f1, Fecha f2)`: devuelve 1 si `f1` es anterior a `f2`, -1 si `f1` es posterior a `f2`, y 0 si son la misma fecha (compara primero el año, luego el mes y por último el día).

Escribe un programa principal que use las fechas 19/2/2024 y 29/2/2024, compruebe si `f1` es bisiesta y válida, y muestre si `f1` es anterior, igual o posterior a `f2`.

Ejercicio 3 — Agenda con TAD Cita

A partir del TAD Fecha de los ejercicios anteriores, define un nuevo TAD Cita compuesto por una Fecha, una hora (entero) y una descripción (cadena de hasta 100 caracteres).

Implementa una agenda como un array de Cita de tamaño máximo `MAX_CITAS` (100), junto con las siguientes funciones:

- `existe_cita(Cita c, Cita agenda[], int tamano)`: devuelve true si ya existe en la agenda una cita con la misma fecha y hora que `c`, y false en caso contrario.
- `anadir_cita(Cita c, Cita agenda[], int *tamano)`: añade la cita `c` a la agenda si hay espacio libre y no existe ya una cita en esa fecha y hora; en caso contrario, informa por pantalla de que no se ha podido añadir (agenda llena) o de que la cita ya existía.

Escribe un programa principal que cree una cita para el 19/2/2024 a las 22h con la descripción "Primera cita", intente añadirla dos veces seguidas a la agenda, y compruebe que la segunda vez se detecta como duplicada.

Ejercicio 4 — Encapsulamiento del TAD Fecha

Reescribe el TAD Fecha del ejercicio 1 aplicando encapsulamiento: los campos día, mes y año pasan a ser private dentro del struct, de modo que no se pueda acceder a ellos directamente desde fuera del tipo.

Añade como funciones miembro del propio struct:

- `crear_fecha(d, m, a)`: inicializa los tres campos privados.
- `obtener_dia()`, `obtener_mes()`, `obtener_anio()`: funciones de acceso (getters) de solo lectura.
- `es_bisiesto()` y `es_valida()`: igual que en el ejercicio 1, pero ahora sin parámetros, ya que operan sobre los campos privados del propio objeto.
- `comparar(Fecha otra)`: equivalente a `comparar_fechas` del ejercicio 2, pero como método del propio TAD.

Adapta el TAD Cita y las funciones `existe_cita` y `anadir_cita` del ejercicio 3 para que utilicen el nuevo TAD Fecha encapsulado (accediendo siempre a través de sus métodos, nunca a sus campos directamente).
Escribe un programa principal equivalente al del ejercicio 3 que demuestre el funcionamiento del TAD encapsulado.